

# SIMTiX: A Compact SIMT Accelerator for RISC-V with a Pipelined FP32/FP16 Datapath

Anonymous Author(s)

Affiliation withheld for double-blind review

**Abstract**—We present SIMTiX, a compact Single-Instruction Multiple-Thread (SIMT) accelerator for RISC-V whose complete path—RTL, simulation, synthesis, place-and-route, and post-route gate-level sign-off—is carried out on a single workstation. Eight lanes grouped into four resident warps attach to a five-stage RISC-V host through a memory-mapped offload interface and implement the full SIMT mechanism set: warp scheduling with a background memory engine, address coalescing, a per-warp reconvergence stack, and an on-chip scratchpad. Placing the vector register file and the scratchpad in distributed RAM reduces LUT usage by  $5.7\times$  and closes 100 MHz timing. Every lane carries a pipelined floating-point unit covering FP32 and FP16 add, multiply, fused multiply-add, and iterative divide and square root, verified bit-exact against an IEEE-754 reference over 116,089 vectors; half precision reuses the single-precision datapath through NaN-boxed widen—compute—narrow at no cycle cost. The central result is a *placed* timing-closure study of this datapath on a Zynq UltraScale+ FPGA. The floating-point fabric is interconnect-bound: we diagnose the limit, confirm it by de-congestion, localize it to a single lane’s fused-multiply-add cone, and close 100 MHz with a three-stage unit that moves the critical path off the floating-point datapath. The complete system-on-chip signs off at 103.7 MHz as a deployable, pin-constrained bitstream with a warning-free design-rule check.

**Index Terms**—SIMT, GPGPU, RISC-V, hardware accelerator, floating-point, half precision, fused multiply-add, timing closure, FPGA, place-and-route.

## I. INTRODUCTION

Graphics processing units execute thousands of scalar threads in lockstep groups called *warps* under a Single-Instruction Multiple-Thread (SIMT) model. The mechanisms that make this efficient—warp scheduling for latency hiding, memory coalescing, control-divergence handling, and on-chip scratchpads—are well documented for commercial GPUs [7], [8], but what they cost once placed and routed is not. Production GPUs are proprietary; open GPGPU cores such as Vortex [1] are powerful but large, and report pre-placement synthesis estimates; and small academic cores are often simulator-only or stop short of a real implementation flow. The result is that the physical behavior of the SIMT mechanism set—which structure actually bounds frequency, and why—is largely unreported.

This paper presents SIMTiX, a deliberately compact SIMT accelerator for RISC-V whose entire path, from RTL through bitstream and post-route gate-level sign-off, is carried out on a single workstation with free or widely available tools. Compactness here is a method rather than a goal. A design small enough to re-place and re-route in full, in hours, is one whose frequency limit can be attributed to a named structure

and then *tested* by changing only that structure and re-running the flow—a controlled experiment that is impractical on a large core and unavailable in a simulator. SIMTiX implements the complete SIMT mechanism set, and every lane is extended with a pipelined floating-point unit. Its headline result is a *placed* timing-closure study showing that, once the lanes carry floating point, the design becomes interconnect-bound rather than logic-bound, and that 100 MHz is reached by attacking the per-lane fused multiply-add (FMA) microarchitecture, a lever identified through a measured diagnose—confirm—localize sequence.

## Contributions.

- A SIMT accelerator (eight lanes, four resident warps) that implements the complete mechanism set—warp scheduling, coalescing, a reconvergence stack, and a scratchpad—in a design compact enough to carry through the entire RTL-to-bitstream flow, integrated with a five-stage RISC-V host as a self-contained system-on-chip (Sections II–III).
- An implementation result showing that moving the vector register file and the scratchpad into distributed RAM reduces LUT usage by  $5.7\times$  and closes 100 MHz timing, with an analysis of why the register-file read multiplexer, not the ALUs, dominates a naive SIMT datapath (Section III).
- A per-lane floating-point extension covering FP32 and FP16, in which half precision is obtained from the single-precision path via NaN-boxed widen—compute—narrow and shown to be correctly single-rounded, verified bit-exact over 116,089 vectors (Section V).
- A placed FPGA timing-closure study that diagnoses the floating-point design as interconnect-bound, confirms the diagnosis by de-congestion, localizes the limit to a single lane’s FMA cone, closes 100 MHz with a three-stage FMA, and carries the closure to the complete system-on-chip at 103.7 MHz with full pin constraints and a warning-free design-rule check (Section VI).

## II. ARCHITECTURE AND PLATFORM

A SIMT machine groups scalar *threads* into *warps* that share one instruction stream and execute on a vector of *lanes*. SIMTiX uses a warp size equal to its lane count ( $\text{WARP\_SIZE} = \text{NUM\_LANES} = 8$ ) with  $\text{NUM\_WARPS} = 4$  warps resident, time-sharing a single fetch/decode/execute datapath; when one warp stalls on memory, the scheduler issues another and hides the latency.

Unlike a packed-SIMD vector unit such as the RISC-V “V” extension [13], where one instruction operates on a register-resident vector, SIMTiX keeps a scalar thread per lane with an independent program-counter abstraction managed by the reconvergence stack, so data-dependent control flow is handled in hardware rather than by masking in software.

**Offload model.** The host programs a command block over a memory-mapped (MMIO) interface—kernel PC, base pointers, and thread count  $N$ —writes a GO bit, and polls DONE. On GO the accelerator launches  $\lceil N/8 \rceil$  warps, seeding each thread’s identifier and arguments through registers at spawn. Four mechanisms make up the platform.

**Warp pool and scheduler.** Four resident slots, each with its own PC, active mask, register-file partition, and reconvergence stack, share one datapath and one data port. A round-robin scheduler issues one runnable warp per cycle, and a background memory engine services a memory access while other warps’ compute instructions keep the datapath busy.

**Coalescing.** The data port is one cache line wide (LINE\_WORDS = 8, 256 bits). The memory engine groups the active lanes’ addresses by line and spends one transaction per distinct line, so a contiguous warp access collapses to a single transaction and a fully scattered one costs up to eight.

**Reconvergence stack.** Each warp owns a SIMT stack whose top-of-stack frame defines the live PC and active mask. A branch on which active lanes disagree pushes a frame so the two lane groups run in turn and reconverge at a join PC, supporting compiler-free single-sided `if` statements and divergent loops.

**Scratchpad.** A per-warp 64-word aperture serviced from on-chip RAM lets data reused across a warp’s lanes be staged once, cutting global traffic.

The active mask additionally drives a per-lane clock enable, so masked-off lanes are not clocked; the unused-lane fraction is dynamic energy that a divergence-aware design saves. From lane utilization measured in our own RTL counters, under a first-order equal-energy-per-lane-instruction model in the spirit of activity-based GPU power models [10], that saving reaches 22.5% of lane-datapath dynamic energy on a heavily divergent kernel. Every mechanism is instrumented with hardware counters and is therefore measured directly in RTL rather than estimated in a software model [9].

### III. IMPLEMENTATION

SIMTiX is written in synthesizable SystemVerilog and integrated with a reused five-stage RV32I host into a self-contained chip (`chip_top`, Fig. 1) comprising the host core, a driver ROM, the accelerator, and an eight-bank on-chip shared memory. The host loads operands into shared memory, launches the accelerator, and reads results back, so the entire CPU→memory→accelerator→memory→CPU loop runs in hardware.

**From flip-flops to LUTRAM.** The first synthesis of the accelerator on a Zynq UltraScale+ (`xczu7ev`) consumed 170,868 LUTs, 74% of the device, at only 10% of its flip-flops. The cause is structural: a flip-flop vector register file

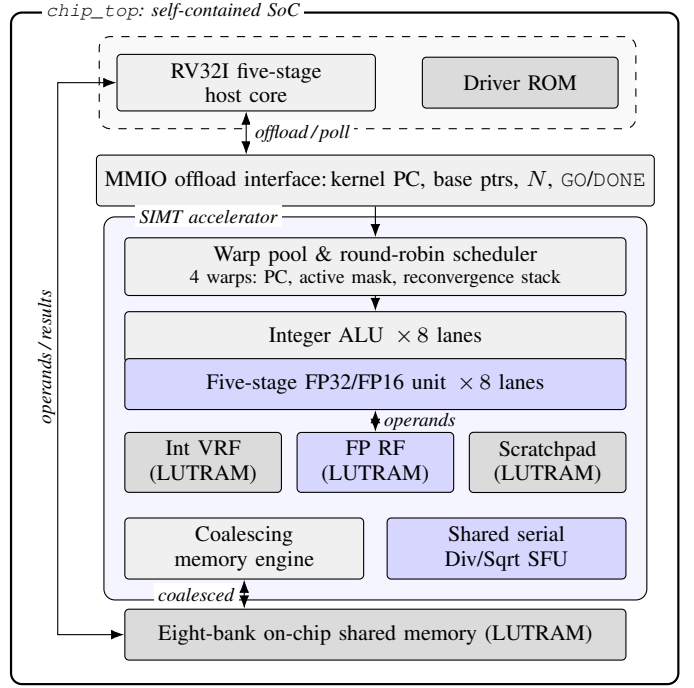


Fig. 1. SIMTiX system architecture. The five-stage RISC-V host drives the SIMT accelerator over a memory-mapped offload interface and exchanges operands and results through the eight-bank on-chip shared memory. Floating-point blocks are tinted blue; register files and the scratchpad live in distributed RAM (LUTRAM). The FP unit is drawn as deployed: the three-stage FMA that closes 100 MHz (Section VI) becomes five stages in the final chip once the significand multiply is fully pipelined onto DSP blocks.

(VRF) of  $4 \times 8 \times 32$  32-bit words is 32,768 registers read as `vrf[warp][lane][reg]` with *both* the warp and the register index variable, that is, a 128-to-1 multiplexer per lane per read port. That multiplexer fabric, not the ALUs, dominates area and sets the critical path. Re-expressing the VRF as one distributed-RAM bank per lane, and likewise the scratchpad, removes essentially all of those registers: LUTs fall  $5.7\times$  to 30,088 (13%), flip-flops to 4,590 (1.0%), and the design closes 100 MHz timing with positive slack at 0.81 W. The complete chip, taken through `opt/place/phys-opt/route` to a bitstream, meets all timing at 100 MHz and uses 34,850 LUTs (15.1%), 6,130 flip-flops, 24 DSP blocks, and zero block RAM at 1.09 W; the accelerator accounts for roughly 87% of the logic.

**Verification ladder.** Correctness is established at four levels: cycle-accurate RTL simulation with self-checking testbenches (Verilator [18]); behavioral simulation in the vendor GUI; full place-and-route with post-route timing sign-off; and post-implementation gate-level functional simulation of the routed netlist (UNISIM primitives). The benchmark suite below adds a fifth, breadth-oriented level.

### IV. PLATFORM EVALUATION

The integer platform is characterized with six kernels chosen to stress distinct mechanisms: `vadd` ( $C_i = A_i + B_i$ , streaming), `saxpy` ( $3A_i + B_i$ , coalescing), `fir` ( $A_i + 2A_{i+1} + A_{i+2}$ ,

TABLE I  
MEASURED CYCLES AND SPEEDUP OVER THE FIVE-STAGE RV32I HOST  
RUNNING IDENTICAL WORK.

| Kernel  | $N$  | Accel.<br>(cycles) | CPU<br>(cycles) | Speedup | Lane util. |
|---------|------|--------------------|-----------------|---------|------------|
| vadd    | 1024 | 1,413              | 13,322          | 9.43×   | 100%       |
| saxpy   | 1024 | 1,669              | 16,394          | 9.82×   | 100%       |
| fir     | 1024 | 2,053              | 14,345          | 6.99×   | 100%       |
| relu    | 1024 | 1,284              | 12,809          | 9.98×   | 93.7%      |
| collatz | 256  | 9,637              | 26,601          | 2.76×   | 68.3%      |
| reduce  | 1024 | 6,023              | 4,363           | 0.72×   | 39.8%      |

stencil), `relu` (light divergence), `collatz` (heavy data-dependent divergence), and `reduce` (scratchpad reduction). Each is assembled from RISC-V source with a stock toolchain, run at  $N \in \{8, 64, 256, 1024\}$  threads, and checked bit-exact against a golden model. For an apples-to-apples speedup, every kernel is also written as a scalar loop and run on the same five-stage host in RTL.

Table I gives the headline platform result. Throughput ramps with occupancy through latency hiding and saturates near 6.7 work-items per cycle, below the eight-lane ideal because of the single shared data port. Streaming kernels reach 7–10× over the scalar host; `fir` is lower (6.99×) because its stencil coalesces only partially; `collatz` achieves 2.76× as heavy divergence erodes, but does not eliminate, the SIMT advantage; and `reduce` falls below break-even at 0.72×, since a single-lane collapse on a single-ported scratchpad is slower than the scalar core and calls for a dedicated cross-lane reduction primitive. SIMTIX is parameterized in its lane and warp counts: a sweep over  $\text{NUM\_LANES} \in \{4, 8, 16, 32\}$  and  $\text{NUM\_WARPS} \in \{2, 4, 8\}$ , in which all twelve configurations elaborate and verify bit-exact, shows throughput scaling near-linearly with lanes ( $\sim 1.97\times$  per doubling), while occupancy beyond two warps pays off only under realistic multi-cycle memory, not under the single-cycle behavioral model used here.

## V. FLOATING-POINT EXTENSION

The base engine is integer-only. It is extended with floating point in both single (FP32) and half (FP16) precision while keeping the same SIMT structure: the extension is a per-lane execute unit and a second per-lane register file, not a new pipeline. Each lane gains a compact fixed-latency FP unit beside its integer ALU, and a second per-lane register file holds the 32 floating-point registers in distributed RAM, addressed by {warp, register} exactly like the integer VRF, with an independent write port so that an FP result and an integer load can retire together. The unit implements add, subtract, multiply, the fused multiply-add in all four sign variants, sign-injection, min/max, the ordered compares, integer $\leftrightarrow$ float and FP32 $\leftrightarrow$ FP16 conversion, the bit-moves, and classify, in both precisions. Rounding is round-to-nearest-even per IEEE 754 [14] and subnormals are flushed to zero, a throughput-oriented choice that keeps each lane small.

**Half precision via widen–compute–narrow.** A 16-bit half lives in a 32-bit register NaN-boxed, with the upper 16 bits all ones; a mis-boxed value reads as a canonical NaN per the RISC-V convention [12]. For an FP16 operation the unit widens the operand exactly to FP32, runs the single-precision datapath, and rounds back to FP16. This is correct rather than approximate: because the FP32 intermediate carries a 24-bit significand and  $24 \geq 2 \cdot 11 + 2$ , rounding first to FP32 and then to FP16 equals rounding directly to FP16 [17], so FP16 add, subtract, multiply, the fused multiply-add, and the conversions are correctly single-rounded. Half precision therefore reuses the verified single-precision logic almost entirely, adding only the widen, narrow, and box steps and, as Fig. 2 shows, no extra cycles.

**Fused multiply-add.** The FMA computes  $\pm(a \cdot b) \pm c$  with a single rounding [15], so it cannot be built by chaining the multiplier and the adder. The unit forms the exact 48-bit product, anchors it in a wide fixed-point accumulator, aligns the addend while capturing a sticky bit, and normalizes and rounds once. When the addend out-ranges the product beyond the rounding window, the product is provably below half a unit in the last place, which yields the addend exactly and bounds the alignment shifter. The FMA has fixed latency and commits through the ordinary writeback path; Section VI describes how this combinational chain is pipelined for timing closure.

**Multi-cycle divide and square root.** Division and square root produce one quotient or root bit per cycle by a digit recurrence [16] and are the engine’s only multi-cycle operations. Because such a core is comparatively large, the engine has a single shared instance: when a warp issues a divide or a square root, the engine captures all of its lanes’ operands and parks the warp on a stall scoreboard, then sequences the active lanes through the shared core one at a time. This is the same park-and-resume mechanism the memory engine uses, so the scheduler keeps issuing other warps onto the idle datapath meanwhile. At most one warp’s divide or square root is in flight, trading latency ( $k$  active lanes take about  $k$  times as long, hidden across warps) for the area saving quantified in Section VI.

**Verification.** Each unit is verified standalone against a DPI-C IEEE-754 reference: FP32 against the host’s correctly-rounded float operations, including `fmaf` and `sqrtof`, and FP16 against a double-based reference cross-checked with `_Float16`. Over random vectors spanning the product-dominant, balanced (cancellation), and addend-dominant regimes, plus directed infinity, NaN, zero, flush, and overflow corners together with divide-by-zero, square-root-of-negative, mis-boxed-operand, and conversion cases, the pipelined unit passes 96,066 checks and the iterative divide/square-root unit a further 20,023, all bit-exact (Table II). End-to-end FP kernels then exercise the full warp pool: streaming add and multiply-add in each precision, a fused  $C_i = A_i B_i + C_i$ , and  $C_i = \sqrt{A_i/B_i}$  with two dependent multi-cycle operations per thread, all bit-exact to 1024 threads.

**Throughput and application kernels.** The FP unit is pipelined (Section VI); an FP instruction commits a fixed

TABLE II  
FLOATING-POINT OPERATION COVERAGE; EVERY OPERATION IS IMPLEMENTED AND CHECKED IN BOTH PRECISIONS.

| Category                                                               | Operations (each in FP32 and FP16)                                         |
|------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Arithmetic                                                             | fadd, fsub, fmul                                                           |
| Fused                                                                  | fmadd, fmsub, fnmsub, fnmadd                                               |
| Divide / root                                                          | fdiv, fsqrt (iterative, multi-cycle)                                       |
| Sign / select                                                          | fsgnj{, n, x}, fmin, fmax                                                  |
| Compare                                                                | feq, flt, fle                                                              |
| Convert                                                                | float $\leftrightarrow$ int (signed/unsigned), FP32 $\leftrightarrow$ FP16 |
| Move / classify                                                        | fmv.x._, fmv._.x, fclass                                                   |
| Bit-exact checks vs. IEEE-754 reference: <b>116,089</b> (0 mismatches) |                                                                            |

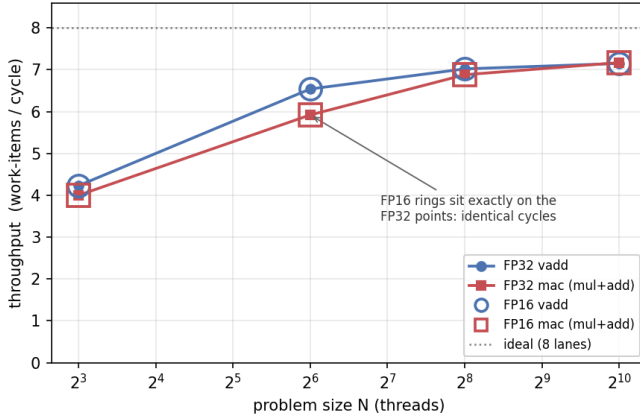


Fig. 2. Floating-point throughput versus problem size for streaming add and multiply-add kernels. The hollow FP16 markers sit exactly on the FP32 curves because both precisions use the same pipelined datapath.

number of cycles after issue and the issuing warp parks a single slot until it does, a latency that SIMT interleaving hides on streaming work. Fig. 2 plots throughput against problem size for streaming add and multiply-add in both precisions. The latency-hiding ramp matches the integer suite and saturates near 7 work-items per cycle at 100% lane utilization, with the FP16 points lying exactly on the FP32 curves because half precision shares the pipelined datapath. Table III runs four toolchain-assembled FP kernels through the engine. Two effects stand out. The iterative divide/square-root unit is visible:  $f_{\text{norm}} = A/\sqrt{B}$  takes about  $6\times$  the cycles of streaming  $f_{\text{saxpy}}$ , the cost of correctly-rounded iterative operations. Divergence also interacts with the now-larger lane datapath:  $f_{\text{dot}}$ 's reduction leaves one of eight lanes active during the collapse, at 45% utilization, so the absolute energy a mask-gated design saves grows with the FP-enlarged lane.

## VI. PLACED TIMING CLOSURE OF THE FP DATAPATH

Floating point raises the accelerator from 30,088 to 74,342 LUTs ( $2.5\times$ ), flip-flops from 4,590 to 7,894, and DSP blocks from 24 to 40 for the FP multipliers, at 1.34 W. The increase is dominated by the common-case arithmetic unit—eight per-lane copies of the add/multiply/FMA/convert datapath, with the fused multiply-add the largest single block—and not by

TABLE III  
FP APPLICATION KERNELS AT  $N=256$  (FP32 UNLESS NOTED), ALL VERIFIED BIT-EXACT.

| Kernel               | Computation                   | Cycles | Gmem | Lane util. |
|----------------------|-------------------------------|--------|------|------------|
| $f_{\text{saxpy}}$   | $\alpha A_i + B_i$ (fmadd)    | 419    | 96   | 100%       |
| $f_{\text{saxpy16}}$ | $\alpha A_i + B_i$ (FP16)     | 451    | 96   | 100%       |
| $f_{\text{norm}}$    | $A_i/\sqrt{B_i}$ (div + sqrt) | 2,396  | 96   | 100%       |
| $f_{\text{dot}}$     | $\sum_l A_{8w+l} B_{8w+l}$    | 1,991  | 96   | 45%        |

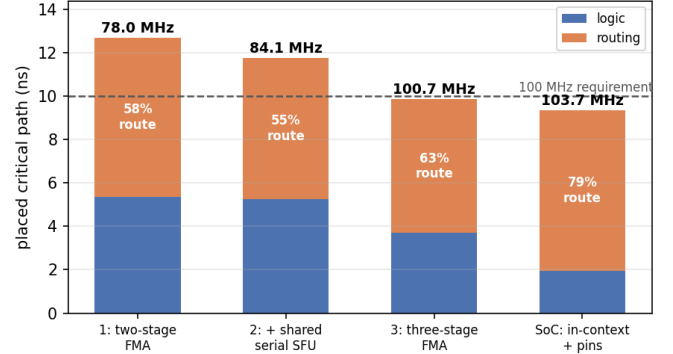


Fig. 3. Placed critical path at each closure step, split into logic and routing delay (worst path of each sign-off timing report, with the achieved  $F_{\text{max}}$  above each bar). Every step is routing-dominated.

the iterative divide/square-root cores, which synthesize to only 689 LUTs each, about 7% of the design. The FMA is also the longest combinational chain: as a single-cycle block its raw path is 16.2 ns (61 MHz), which makes it the frequency bottleneck.

Table IV summarizes the placed utilization and frequency of every step in the closure. Steps 1–3 are out-of-context runs holding the lane count at eight, the DSE row trims the design to four lanes, and the final row is the complete system-on-chip implemented in-context with full pin constraints after the fetch/execute split described at the end of this section. Fig. 3 plots the placed critical path at each step, split into logic and routing delay; the routing share never drops below half.

**Step 1: two-stage FMA, 78 MHz.** The FP unit is first pipelined into two stages, split at the FMA accumulator (mag) boundary. The warp pool absorbs the extra cycle with a single-slot scoreboard ( $W_{\text{FPC}}$ ), the same park-and-resume mechanism used for memory and the SFU, so no new structural hazard arises and SIMT interleaving hides the latency. Timing-driven synthesis with retiming cuts the FP logic depth to 5.3 ns, yet place-and-route closes at 12.8 ns (78 MHz). The placed critical path is still the first FMA stage but is now routing-dominated, with 7.4 of 12.8 ns (58%) in wire delay, because in the FP-enlarged fabric ( $2.5\times$  the integer LUTs across eight wide FP lanes) interconnect rather than logic depth has become the binding constraint.

**Step 2: confirming the diagnosis, 84 MHz.** If congestion is the limit, de-congesting the fabric should raise frequency with the logic unchanged. The eight per-lane divide/square-

TABLE IV  
PLACED POST-ROUTE RESULTS ACROSS THE TIMING-CLOSURE STEPS, THE FOUR-LANE DSE POINT, AND THE FINAL DEPLOYABLE SYSTEM-ON-CHIP.

| # | Configuration               | Lanes | LUT    | LUT% | LUTRAM | FF     | DSP | Power (W) | $F_{\max}$ (MHz) |
|---|-----------------------------|-------|--------|------|--------|--------|-----|-----------|------------------|
| 1 | Two-stage FMA               | 8     | 72,573 | 31.5 | 3,348  | 12,347 | 40  | 1.53      | 78.0             |
| 2 | + shared serial SFU         | 8     | 68,914 | 29.9 | 3,348  | 11,153 | 40  | 1.22      | 84.1             |
| 3 | + three-stage FMA           | 8     | 69,987 | 30.4 | 3,348  | 14,954 | 40  | 1.22      | <b>100.7</b>     |
|   | DSE point: four lanes       | 4     | 36,785 | 16.0 | 1,748  | 6,953  | 20  | 0.98      | 79.7             |
|   | Deployable SoC (in-context) | 8     | 71,567 | 31.1 | 7,244  | 17,834 | 40  | 1.20      | <b>103.7</b>     |

Target `xczu7ev` (230,400 LUTs, 460,800 FFs, 1,728 DSPs) at a 100 MHz constraint; every build uses zero block RAM. LUT% is of the device LUT count; LUTRAM is the distributed-RAM subset of the LUT column and, in the SoC row, includes the eight-bank shared memory. Placed critical path by row: the FMA cone in rows 1, 2 and at the DSE point; the VRF LUTRAM read multiplexer in row 3; the FP control broadcast in the SoC row.

root cores are folded into a single shared serial unit fed lane by lane. This removes about 3.7k LUTs and 1.2k flip-flops and drops power from 1.53 to 1.22 W; crucially it leaves the FMA critical path structurally unchanged yet cuts that path’s routing delay from 7.4 to 6.5 ns, raising the placed maximum to 84 MHz and reducing near-critical endpoints from 8.7k to 3.2k. Same logic, less congestion, higher frequency: a direct confirmation that interconnect, not logic depth, bounds this design.

**Localizing the limit: frequency does not scale with fabric size.** A natural question is whether trimming the FP fabric relieves congestion further. It does not. Placing the design at four lanes (Table IV, DSE row) halves its area, 36,785 versus 68,914 LUTs and 20 versus 40 DSP blocks, but reaches only 79.7 MHz, marginally below the eight-lane 84.1 MHz, and the router still reports CLB congestion. The critical path is a single lane’s FMA cone, whose logic depth and locally dense 128-bit datapath are independent of the lane count; halving the lanes replicates that cone fewer times but packs the smaller design into a denser region that gives the lone FMA no relief. The eight-lane configuration therefore Pareto-dominates the four-lane one, at equal frequency and twice the throughput, and identifies the true frequency lever as the FMA microarchitecture rather than the fabric size.

**Step 3: closing 100 MHz with a three-stage FMA.** Acting on that finding, the FMA is split across three stages—product, 128-bit align-and-add, and normalize-and-round—so that no single stage carries the whole cone. The warp pool absorbs the extra cycle with the same single-slot scoreboard, now a two-cycle writeback latency that SIMT interleaving continues to hide on streaming work. The added registers raise flip-flops to 14,954 while leaving LUTs (69,987), DSP blocks, power (1.22 W), and the bit-exact results unchanged. Place-and-route now meets timing at 100 MHz with a worst setup slack of +0.071 ns, tracing 78 → 84 → 100.7 MHz across the three steps, and the critical path moves off the FMA onto the vector-register-file LUTRAM read multiplexer, the same structure that bounds the integer design (Section III). The floating-point datapath is no longer the frequency limiter, which confirms that FMA logic depth—the lever the design-space exploration identified—was the true constraint.

**From out-of-context closure to a deployable SoC.** Closing

the accelerator out-of-context is not yet a chip. Integrated into `chip_top` with the host core and the eight-bank shared memory, the FP-enlarged SoC first places at only 82.5 MHz. The worst path is again congestion-bound, but now through the warp scheduler’s fetch–execute cloud: instruction fetch from the asynchronous LUTRAM memory, decode, register-file read, execute, and writeback all completed in one combinational cycle, and 82% of the placed path was routing on its two highest-fanout broadcast nets. Registering FP operands at issue, replicating the broadcast nets, and fully pipelining the DSP multiplies recover part of the gap (96.4 MHz) but leave the single-cycle cloud intact. The last of those changes also deepens the FP unit: `fmul` and the FMA need the same 48-bit significand product, so it is computed once on a *shared*  $24 \times 24$  DSP cascade carrying a full input register and three output registers, the pattern that draws no DSP pipelining advisories. That places the product four cycles after the operands and makes the unit in the deployed chip five stages deep rather than three; the stage-0 payload rides a matching delay line, the arithmetic is untouched and every bit-exact result unchanged, and the single-slot `W_FPC` scoreboard counts out the longer latency. Splitting the fetch–execute cloud with a fetch/execute pipeline register—the fetched instruction and its warp context are registered at a new stage boundary, and a per-warp in-flight bit preserves the scheduler’s re-issue semantics exactly—applies the SIMT latency-hiding argument at instruction grain: with two or more runnable warps the pipeline stays full, and the measured cycle cost is at most 0.3% on 1024-element kernels. The full chip then places at 107.3 MHz unconstrained and, with every port pin-constrained for the ZCU104 (clock on a clock-capable pin) and a warning-free design-rule check with no waivers or severity downgrades, signs off at +0.360 ns worst setup slack (103.7 MHz), 71,567 LUTs (31%), 40 DSP blocks, zero block RAM, and 1.20 W (Table IV, final row), yielding a deployable bitstream.

## VII. RELATED WORK

Open RISC-V GPGPU cores include Vortex [1], a full OpenCL-capable multi-core design, and Ventus [2], a PTX/Vortex-style GPGPU; both target performance and scale and are correspondingly large. Simty [3] demonstrates generalized SIMT execution of the RISC-V ISA, integer-only and re-

TABLE V  
VORTEX PER-CORE NUMBERS NEXT TO THE COMPLETE SIMTiX SoCs.

| Design          | W×L | FP      | LUT   | FF    | BRAM | MHz                |
|-----------------|-----|---------|-------|-------|------|--------------------|
| Vortex [1]      | 4×4 | FP32    | 21.5k | 32.7k | 131  | 233 <sup>†</sup>   |
| Vortex [1]      | 4×8 | FP32    | 37.9k | 57.6k | 247  | 224 <sup>†</sup>   |
| SIMTiX, integer | 4×8 | —       | 34.9k | 6.1k  | 0    | 100 <sup>‡</sup>   |
| SIMTiX, FP      | 4×8 | FP32/16 | 71.6k | 17.8k | 0    | 103.7 <sup>‡</sup> |

W×L denotes resident warps × lanes. <sup>†</sup> Synthesis estimate on an Intel fabric, from [1], Table 3; each Vortex core includes its L1 caches and shared memory (BRAM column). The demonstrated Vortex system in [1] runs at 200 MHz. <sup>‡</sup> Post-route on UltraScale+, for the complete SoC including the host CPU, driver ROM, and shared memory, against a 100 MHz target met with positive slack.

porting synthesis area in logic elements on an Intel Cyclone IV, and Nyuzi (published as Nyami) [4] is a synthesizable vector GPGPU for graphics research.

Table V places SIMTiX next to the per-core FPGA numbers published for Vortex, the closest comparable design: the complete SIMTiX SoC, host CPU, driver ROM, and eight-bank shared memory included, fits in the logic budget of roughly one Vortex core of equal warp and lane geometry. The frequency columns, however, are not directly comparable, and we do not read the gap as a two-fold speed deficit. The Vortex entries are pre-placement synthesis estimates on an Intel fabric; the system actually demonstrated in [1] runs at 200 MHz. The SIMTiX entries are post-route on UltraScale+ and are bounded by the 100 MHz target we constrain the build to rather than by a swept maximum: both SoCs close that target with positive slack, so the reported figure is a signed-off operating point, not a ceiling. The wider point is that a pre-placement number says little about the placed one: Section VI reports our FP design synthesizing to a 5.3 ns logic depth yet closing at 12.8 ns placed, because the binding constraint turned out to be routing congestion rather than logic depth. That placed, signed-off closure study is not reported for the larger cores. Open floating-point units such as FPnew [11] offer multifunction arithmetic as standalone IP; SIMTiX instead integrates a per-lane FP datapath into the SIMT pipeline and studies its placed timing. Control-divergence handling via reconvergence stacks is long studied [5], [6]; SIMTiX uses the standard stack mechanism and reuses its mask for lane clock-gating.

### VIII. CONCLUSION

SIMTiX shows that the full SIMT mechanism set fits in a compact RISC-V accelerator that runs end-to-end through a real implementation flow, with a distributed-RAM register file the key to making it small and fast on FPGA, and a per-lane unit adding FP32 and FP16 arithmetic verified bit-exact over 116,089 vectors. The central result is a placed timing-closure study: the FP-enlarged fabric is interconnect-bound, a limit diagnosed, confirmed by de-congestion, and localized to a single lane’s FMA cone, then closed at 100 MHz with a three-stage FMA that moves the critical path onto the same register-file read multiplexer that bounds the integer design. A

fetch/execute pipeline in the warp scheduler carries that closure to the complete SoC, which signs off at 103.7 MHz with full ZCU104 pin constraints and a warning-free design-rule check, at a measured cycle cost under 0.3%. An ASIC path through the SkyWater 130 nm PDK [19] with OpenLane [20] would turn these FPGA numbers into silicon-relevant area, frequency, and power.

### ACKNOWLEDGMENT

Per the conference policy on artificial intelligence, the authors declare that a large language model (Anthropic Claude) assisted with the preparation of this manuscript and with supporting build and analysis scripting. The RTL and testbench code, and all design and experimental decisions, were the authors’ own; every reported result was produced by the tools named in the text and checked by the authors, who take full responsibility for this paper.

### REFERENCES

- [1] B. Tine, K. P. Yalamarthy, F. Elsabbagh, and H. Kim, “Vortex: Extending the RISC-V ISA for GPGPU and 3D-Graphics,” in *Proc. IEEE/ACM MICRO*, 2021.
- [2] F. Zhang *et al.*, “Ventus: a RISC-V based GPGPU,” open-source project, 2022.
- [3] S. Collange, “Simty: Generalized SIMT execution on RISC-V,” in *Workshop on Computer Architecture Research with RISC-V (CARRV)*, 2017.
- [4] J. Bush, P. Dexter, T. N. Miller, and A. Carpenter, “Nyami: a synthesizable GPU architectural model for general-purpose and graphics-specific workloads,” in *Proc. IEEE ISPASS*, 2015.
- [5] W. W. L. Fung, I. Sham, G. Yuan, and T. M. Aamodt, “Dynamic Warp Formation and Scheduling for Efficient GPU Control Flow,” in *Proc. IEEE/ACM MICRO*, 2007.
- [6] E. Lindholm, J. Nickolls, S. Oberman, and J. Montrym, “NVIDIA Tesla: A Unified Graphics and Computing Architecture,” *IEEE Micro*, vol. 28, no. 2, 2008.
- [7] T. M. Aamodt, W. W. L. Fung, and T. G. Rogers, *General-Purpose Graphics Processor Architectures*. Morgan & Claypool, 2018.
- [8] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017.
- [9] A. Bakhoda, G. L. Yuan, W. W. L. Fung, H. Wong, and T. M. Aamodt, “Analyzing CUDA Workloads Using a Detailed GPU Simulator,” in *Proc. IEEE ISPASS*, 2009.
- [10] J. Leng *et al.*, “GPUWattch: Enabling Energy Optimizations in GPGPUs,” in *Proc. ACM/IEEE ISCA*, 2013.
- [11] S. Mach, F. Schuiki, F. Zaruba, and L. Benini, “FPnew: An Open-Source Multifunction Floating-Point Unit Architecture for Energy-Proportional Transprecision Computing,” *IEEE Trans. VLSI Syst.*, vol. 29, no. 4, pp. 774–787, 2021.
- [12] A. Waterman and K. Asanović, “The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA,” RISC-V International, 2019.
- [13] RISC-V International, “RISC-V “V” Vector Extension, Version 1.0,” specification, 2021.
- [14] IEEE, “IEEE Standard for Floating-Point Arithmetic,” *IEEE Std 754-2019*, 2019.
- [15] J.-M. Muller *et al.*, *Handbook of Floating-Point Arithmetic*, 2nd ed. Birkhäuser, 2018.
- [16] M. D. Ercegovac and T. Lang, *Digital Arithmetic*. Morgan Kaufmann, 2004.
- [17] S. A. Figueroa, “When is double rounding innocuous?,” *ACM SIGNUM Newsletter*, vol. 30, no. 3, pp. 21–26, 1995.
- [18] W. Snyder, “Verilator: Open-Source SystemVerilog Simulation and Verification,” Veripool, <https://www.veripool.org/verilator>.
- [19] SkyWater Technology and Google, “SkyWater Open Source PDK (sky130),” <https://github.com/google/skywater-pdk>.
- [20] A. Ghazy and M. Shalan, “OpenLane: The Open-Source Digital ASIC Implementation Flow,” in *Workshop on Open-Source EDA Technology (WOSET)*, 2020.